Controlled Latent Cognitive Dynamics: A Falsifiable Research Program for Treating Machine Reasoning as a Controllable Latent Trajectory

Ioannis Vandoulas
Independent Researcher
jvandoul@gmail.com

July 2026 — Draft v0.2 (position paper; no empirical results yet)

Abstract

Current methods for improving reasoning in large language models operate almost exclusively at the token level: chain-of-thought prompting, tree-structured search over textual states, self-reflection, and multi-agent debate. We propose Controlled Latent Cognitive Dynamics (CLCD), a research program that instead treats machine reasoning as a trajectory through latent state space, where reusable state-dependent operators transform the current state and a higher-level controller selects among them online, based on geometric statistics of the recent trajectory. The program combines an explicit operator alphabet (EXPAND, CHALLENGE, CONNECT, CONSOLIDATE), a fully specified extraction-and-injection mechanism for activation-level intervention, a trajectory regime metric with pre-registered target zones, and a controller that optimizes trajectory dynamics rather than only final outputs. We make the epistemic status of every component explicit: results obtained while operators are implemented through prompts are observational and support only claims about adaptive prompt scheduling; the central claim of latent control requires causal, intervention-level evidence. We further define a trajectory-first discovery path in which operators are learned as functional equivalence classes from a large Trajectory Atlas rather than imposed a priori, guarded against circular evaluation, prompt artifacts, and compute confounds by an inherited measurement protocol. Six hypotheses, ten falsification conditions, and staged decision gates define how the program fails. Complete technical specifications are included as appendices.

1 Introduction

Progress in machine reasoning over the past several years has been driven largely by structured manipulation of text: eliciting intermediate reasoning tokens [15], searching over trees or graphs of textual thought states [7, 8], iterative self-critique [9, 10], and debate between model instances [11]. These methods share a common substrate — the reasoning state they inspect and control is decoded text — and a common cost profile: every control decision requires generating and re-reading tokens.

In parallel, work on activation steering and representation engineering [1, 2, 3, 4] has established a different fact: adding relatively simple vectors to a transformer's residual

stream reliably and causally shifts model behavior along interpretable directions, and compact function vectors can carry task identity across contexts [5, 6]. Latent interventions are therefore causally potent. Yet this line of work has largely produced open-loop, single-direction interventions: a vector is chosen offline and applied uniformly, with no notion of when a given intervention is appropriate.

Controlled Latent Cognitive Dynamics (CLCD) is a research program at the intersection of these two lines. Its central architectural claim is that machine reasoning can be modeled and controlled as a trajectory through latent state space, where reusable state-dependent operators — organized into a small alphabet defined by reasoning function rather than by content — transform the current state, and a higher-level controller selects among them online in order to keep the trajectory inside a productive dynamical regime. The claim is deliberately operational: a latent state is treated as a representation that can be observed, compared, transformed, and evaluated, with no assumption that it corresponds to a complete thought or an interpretable semantic unit.

This paper is a position and program paper: it reports no empirical results. Its contribution is a fully specified, falsifiable engineering proposal — including the component that proposals of this kind usually leave undefined (how a modified state is injected back into the model), the measurement discipline needed to avoid the standard failure modes of the genre (circular judging, prompt artifacts, compute confounds, post-hoc metric fitting), and an explicit account of what evidence would count against the program. The complete technical specifications are included verbatim as Appendix A (architecture) and Appendix B (trajectory-first research plan).

2 Motivation

Three observations motivate the program. First, token-level scaffolding is expensive and opaque as a control interface: the controller (usually a prompt schedule or a search heuristic) can only read decoded text, so every measurement of the reasoning state costs generation, and the state it measures is a lossy verbalization. If reasoning-relevant structure exists in activations — as the steering literature suggests — a controller that reads geometric trajectory statistics directly could act earlier, more cheaply, and on signals that never surface in text.

Second, existing latent interventions are open-loop. Steering vectors demonstrate causal efficacy but not policy: nothing in that literature selects, at step t, which functional transformation the current reasoning trajectory needs. The natural missing component is closed-loop control — a policy over a small set of functional operators, conditioned on where the trajectory currently is and how it has been moving.

Third, productive reasoning plausibly lives in an intermediate dynamical regime. A trajectory that barely moves is stagnating; one that moves maximally with low coherence is drifting. CLCD formalizes this intuition as a measurable, pre-registered trajectory-regime hypothesis rather than leaving it as metaphor: a regime metric Ω combining transition magnitude, directional diversity, and task coherence, with target zones fixed before confirmatory evaluation. If the data do not support a productive zone, the construct is removed by an explicit decision gate.

3 Related Work

3.1 Activation steering and representation engineering

Latent steering vectors [1], activation addition [2], contrastive activation addition [3], representation engineering [4], function vectors [5], and in-context vectors [6] establish that fixed residual-stream interventions causally shift behavior. CLCD treats this as prior art rather than as a contribution: what it adds is (a) an operator alphabet defined by reasoning function, (b) state-dependent rather than fixed transformations, and (c) online selection by a controller reading trajectory geometry. The open question CLCD targets is whether steering-style intervention can reproduce operator-specific reasoning functions on demand — not whether steering works at all.

3.2 Structured reasoning search

Chain-of-thought [15], Tree of Thoughts [7], Graph of Thoughts [8], Self-Refine [9], Reflexion [10], multi-agent debate [11], meta-prompting [12], and Self-Discover [13] search or schedule over discrete reasoning moves expressed in text, with evaluators scoring intermediate textual states. CLCD differs in the substrate (latent vectors, not text), in the controller inputs (geometric trajectory statistics, not text-level self-evaluation), and — at its causal stages — in the actuation mechanism (direct activation intervention, not prompting). An important honesty constraint follows: while CLCD operators are implemented through prompts (its Stage 1), the system is a member of this family, and results must be reported as adaptive prompt scheduling.

3.3 Latent-space reasoning

Coconut [14] and related work carry reasoning steps in continuous states without decoding to text, typically as end-to-end learned latent rollouts. CLCD adds an explicit, discrete, human-auditable operator alphabet over such states, a controller, and a trajectory-level objective — i.e., structure and control rather than free latent recurrence.

3.4 Options and skill discovery; latent dynamics analysis

The trajectory-first discovery path (Appendix B) is structurally an options / skill discovery program in the sense of hierarchical reinforcement learning [16, 17]: discover reusable transformations from experience, then learn a policy over them. The difference is the environment — the model's own latent reasoning space — and the validity criterion, which is a functional effect profile on reasoning outcomes. The analysis toolset for discovering recurring dynamical modes (switching linear dynamical systems, state-space segmentation [18]) is standard in computational neuroscience, whose central pitfall — descriptive dynamical structure without causal validation — CLCD encodes as explicit decision gates.

4 Contributions

The program makes five commitments, each specified in full in the appendices.

(1) A complete control architecture. Operational latent states $s_t = P(H_t)$ with mandatory geometric normalization; transition vectors δ_t; a four-operator alphabet with textual prototypes and paraphrase controls; a transition generator; a state evaluator; and a rule-based controller over trajectory statistics (magnitude, directional diversity, coherence, repetition, candidate quality, operator history). Critically, the architecture specifies not only state *extraction* but state *injection*: a two-step mechanism (de-normalize, then lift through the encoder's pseudo-inverse into the residual stream) with reconstruction and fluency-preservation validity checks, and explicit failure semantics separating “inadequate encoder/injection pair” from “no operator geometry.”

(Appendix A, Sections 4–6.)

(2) An explicit epistemic ladder. Every result is labeled with the causal status of the state under which it was obtained. In prompt-implemented stages the latent state is a post-hoc summary that does not drive generation; such results are observational and support scheduling claims only. The central claim of latent control requires intervention-level evidence in which the state is on the causal path to the output. Hypotheses are stage-qualified accordingly (H4-S1 vs. H4-S3). (Appendix A, Section 4.3.)

(3) A measurement discipline. A metric registry typing every quantity as geometric, embedding-based, judge-based, or objective; a judge-separation protocol preventing the same evaluator from driving both inner-loop selection and outcome evaluation; two-ledger compute accounting (model tokens vs. total cost including controller overhead) with matching on realized rather than nominal budgets; equal hyperparameter tuning budgets across all compared conditions; and pre-registration of regime-zone parameters and minimum effect sizes. (Appendix A, Sections 8, 17, 20.)

(4) A trajectory-first discovery path. Rather than assuming the four-operator alphabet, the program's first experiment is a Trajectory Atlas: a large observational map of latent reasoning trajectories from diverse sources (with a mandated floor of non-operator-prompted episodes). Operators are then discovered as functional equivalence classes — clusters stable across seeds, paraphrases, domains, step definitions, and distance-weight settings, with demonstrated seed-label independence — and only then converted into causal transformations and a closed-loop controller. The named operators are demoted to seed intervention families. (Appendix B.)

(5) Falsifiability. Six hypotheses (operator separability; cross-domain transfer; causal intervention; controller advantage under equal realized compute; a productive regime zone; mediation of controller gains by trajectory dynamics), ten falsification conditions, and staged decision gates that specify, in advance, which component is rejected or revised under each negative outcome.

5 Method Overview

5.1 States, transitions, and operators

A latent state $s_t \in R^d$ is extracted from hidden activations by a fixed pooling/projection function and mapped into a normalized space (whitening or standardization frozen on a calibration split) before any geometry is computed; key results must replicate under at least two normalization choices. The elementary observation is the transition $\delta_t = s_{t+1} - s_t$, with magnitude and unit direction as primitive features. An operator $T_a : S \to S$ is a transformation intended to implement a reasoning function; the initial alphabet is EXPAND (increase search breadth), CHALLENGE (destabilize unsupported structure), CONNECT (create relational structure), CONSOLIDATE (restore coherence and compress). An operator is validated only by repeatable, causally useful transformation across contexts — never by its prompt wording.

5.2 Injection: from modified state back into the model

An operator effect Δ estimated in normalized space is applied in two explicit steps: $\Delta_{raw} = W^{-1}\Delta$ (the centering term cancels for differences), then residual-stream addition $h' = h + \alpha \cdot P^{+}(\Delta_{raw})$ at specified layers, token positions, and schedule, with strength α calibrated against naturally observed transition norms. Interventions must pass

reconstruction-sanity and fluency-preservation checks before any behavioral conclusion is drawn.

5.3 Controller and trajectory regime

The controller Π selects the next operator from a summary z_t of the recent trajectory: mean transition magnitude M_t, directional diversity V_t, coherence with the task C_t, repetition ρ_t, candidate-hypothesis quality Q_t, and operator history. A rule-based v0.2 policy handles stagnation, premature agreement, instability, fragmentation, candidate maturity, and stopping; all thresholds fall under the equal-tuning-budget protocol. The regime metric $\Omega_t = g(M_t)\cdot h(V_t)\cdot C_t$ encodes the target-zone hypothesis; its parameters and the productive-zone bounds are pre-registered or selected by cross-validation and evaluated once on held-out data.

5.4 Hypotheses and experiments

H1 (separability): operator-conditioned transitions are distinguishable under cross-domain and cross-paraphrase splits. H2 (transfer): learned operator transformations preserve functional effects in unseen domains. H3 (causal intervention): injected transformations reproduce the intended reasoning function without textual instruction. H4 (controller advantage): dynamic operator selection beats fixed cycles and standard QA under equal realized compute — stage-qualified as scheduling (H4-S1) vs. latent control (H4-S3). H5 (productive zone): high-quality outcomes concentrate inside the pre-registered Ω region. H6 (mediation): controller gains are partly mediated by trajectory dynamics rather than extra computation. The experimental sequence runs from an operator-discovery study (separability, transfer, intervention, downstream utility, mediation) against baselines including chain-of-thought, tree search, debate, and — most importantly — an equal-compute random operator policy.

5.5 The trajectory-first program

Appendix B inverts the order of assumptions: Phase A builds the Trajectory Atlas (≥ 4 domains, ≥ 10,000 episodes, successful and unsuccessful reasoning, ≥ 30% non-operator sources, two step definitions); Phase B discovers functional modes via a functional distance combining geometry, effect profiles, and context, with weight-grid sensitivity and seed-label independence requirements; Phase C converts stable modes into causal operators against random-direction, PCA-direction, and shuffled-label controls; Phase D learns outcome models over trajectory features with length and difficulty confound controls; Phase E closes the loop with a learned controller under three-way data splits separating regime discovery, controller training, and evaluation. A parallel low-cost track runs the seed-operator separability experiment on Atlas data from day one, de-risking the slower discovery path.

6 Discussion

The program's value does not depend on its central claim being true. Each decision gate produces information: if operator transitions are not separable beyond prompt artifacts, that constrains what pooled activation summaries carry; if geometry is separable but not causally reproducible, the field learns that footprints of reasoning functions exist without being controllable handles; if intervention works but closed-loop selection adds nothing over random schedules under matched compute, latent control collapses into steering plus computation. The sharpest risk to the genre — declaring victory on gains that are actually extra compute, judge familiarity, or post-hoc metric

fitting — is addressed structurally rather than by good intentions: realized-compute matching in two ledgers, judge separation, pre-registration, and equal tuning budgets are binding protocol, not discussion-section caveats.

The two documents play complementary roles. The architecture specification (Appendix A) fixes interfaces precisely enough to implement; the trajectory-first plan (Appendix B) prevents the architecture's own vocabulary from contaminating discovery, by demoting the named operators to data-generation instruments and requiring that any discovered taxonomy demonstrate independence from its seeds. If the discovered modes simply recover EXPAND / CHALLENGE / CONNECT / CONSOLIDATE, that is a validation of the alphabet; if they cut across it, the alphabet is revised; if nothing stable is discovered under an adequate representation sweep, the operator hypothesis itself is unsupported.

7 Limitations

First and foremost, this is a proposal: no experiment has been run, and every mechanism described here may fail at its first gate. Second, a single pooled vector is a severely lossy summary of a reasoning episode whose full causal state is the entire context and KV cache; the program mitigates this by treating encoders, normalizations, and step definitions as swappable representations requiring robustness sweeps, but an inadequate-representation outcome remains likely. Third, several constructs (hypothesis quality, testability) are judge-based even under separation protocols, and judge noise bounds what the inner loop can optimize. Fourth, results may be model-family-specific; cross-model transfer is explicitly deferred. Fifth, the difference-vector formalism assumes locally linear structure in the normalized space, which is a modeling choice with a declared fallback rather than a validated property. Finally, the Trajectory Atlas is data-hungry, and the discovery phases carry a real risk of remaining exploratory; the parallel separability track exists precisely to bound that risk.

8 Future Work

The immediate path is the minimum prototype of Appendix A Section 26 and Appendix B Section 18: one open-weight model with hidden-state access, one frozen encoder with two normalization variants and two step definitions, a hidden-solution task set with difficulty labels, full trajectory logging, and the parallel separability experiment. Contingent on gate outcomes, subsequent work includes learned operator modules, a learned controller, joint training of encoder, operators, controller and stopping policy, an emergent operator alphabet, cross-model operator transfer, and the deferred unification of internal and external dialogue as transition sources. Negative results will be published with the same care as positive ones.

References

[1] Subramani, N., Suresh, N., Peters, M. (2022). Extracting Latent Steering Vectors from Pretrained Language Models. Findings of ACL 2022.

[2] Turner, A., Thiergart, L., Udell, D., Leech, G., Mini, U., MacDiarmid, M. (2023). Activation Addition: Steering Language Models Without Optimization. arXiv:2308.10248.

[3] Rimsky, N., Gabrieli, N., Schulz, J., Tong, M., Hubinger, E., Turner, A. (2024). Steering Llama 2 via Contrastive Activation Addition. ACL 2024.

[4] Zou, A., et al. (2023). Representation Engineering: A Top-Down Approach to AI Transparency. arXiv:2310.01405.

[5] Todd, E., Li, M., Sharma, A., Mueller, A., Wallace, B., Bau, D. (2024). Function Vectors in Large Language Models. ICLR 2024.

[6] Liu, S., Ye, H., Xing, L., Zou, J. (2023). In-Context Vectors: Making In-Context Learning More Effective and Controllable Through Latent Space Steering. arXiv:2311.06668.

[7] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., Narasimhan, K. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. NeurIPS 2023.

[8] Besta, M., et al. (2024). Graph of Thoughts: Solving Elaborate Problems with Large Language Models. AAAI 2024.

[9] Madaan, A., et al. (2023). Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS 2023.

[10] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 2023.

[11] Du, Y., Li, S., Torralba, A., Tenenbaum, J., Mordatch, I. (2024). Improving Factuality and Reasoning in Language Models through Multiagent Debate. ICML 2024.

[12] Suzgun, M., Kalai, A. (2024). Meta-Prompting: Enhancing Language Models with Task-Agnostic Scaffolding. arXiv:2401.12954.

[13] Zhou, P., et al. (2024). Self-Discover: Large Language Models Self-Compose Reasoning Structures. NeurIPS 2024.

[14] Hao, S., Sukhbaatar, S., Su, D., Li, X., Hu, Z., Weston, J., Tian, Y. (2024). Training Large Language Models to Reason in a Continuous Latent Space (Coconut). arXiv:2412.06769.

[15] Wei, J., et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022.

[16] Sutton, R., Precup, D., Singh, S. (1999). Between MDPs and semi-MDPs: A Framework for Temporal Abstraction in Reinforcement Learning. Artificial Intelligence, 112(1-2).

[17] Eysenbach, B., Gupta, A., Ibarz, J., Levine, S. (2019). Diversity is All You Need: Learning Skills without a Reward Function. ICLR 2019.

[18] Linderman, S., Johnson, M., Miller, A., Adams, R., Blei, D., Paninski, L. (2017). Bayesian Learning and Inference in Recurrent Switching Linear Dynamical Systems. AISTATS 2017.

Appendix A — CLCD Architecture Specification v0.2 (verbatim)

```
CLCD Architecture Specification v0.2
====================================

Controlled Latent Cognitive Dynamics
(method sections use the neutral term: Latent Trajectory Control)

Status
------
Working technical research specification.
This document defines a minimum implementable architecture for testing whether
machine reasoning can be controlled as a trajectory through latent state space.

This is not yet a validated theory, a production architecture, or a claim of human-like
cognition. It is a falsifiable engineering proposal.

Version
-------
v0.2

Changes from v0.1 (summary)
---------------------------
1.  Added Section 3 (Related Work and Novelty Differentiation); the novelty claim
    is now stated relative to specific prior lines of work.
2.  Added Section 6 (State Injection Mechanism). v0.1 defined extraction P but not
    the inverse pathway; all intervention experiments depend on it.
3.  Added Section 4.3 (Causal Status of the State by Stage). The observational role
    of s_t in Stages 1-2 versus its causal role in Stages 3+ is now explicit, and
    hypothesis H4 is stage-qualified accordingly.
4.  Added Section 4.2 (Geometry Requirements): whitening / normalization is now
    mandatory before any norm- or cosine-based trajectory metric.
5.  Added Section 9 (Measurement Definitions): operational definitions for
    coherence, hypothesis quality, hypothesis potential, testability, and
    repetition, plus a judge-separation protocol to prevent evaluator circularity.
6.  Ω ("Creative Tension") renamed to Trajectory Regime Metric in method sections;
    components are normalized; the H5 target zone must be pre-registered or
    selected by cross-validation on held-out data (Section 12.2).
7.  Added Section 17 (Statistical Analysis and Compute Accounting): seeds, tests,
    multiple-comparison correction, minimum effect sizes, dataset sizing, and an
    explicit definition of "equal compute" that includes controller overhead.
8.  Baseline protocol now requires equal hyperparameter tuning budget (Section 20).
9.  Symbol hygiene: repetition is ρ_t (was R_t), restructuring is Ψ_t (was R_t),
    testability is U(s) (was T(s)), operator classifier is f_cls (was C).
10. Controller summary state z_t now includes operator history (consistent with
    the module interface).
11. v0.1 Section 10 (Internal and External Dialogue) removed from the core spec
    and deferred to Future Work (Section 27); it was underdeveloped and is not
    needed for any v0.2 experiment.

Research Lineage
----------------
DPHG  ->  MVCM  ->  CLCD

DPHG:
Dialogue Protocol for Hypothesis Generation

MVCM:
Minimum Viable Cognitive Model

CLCD:
Controlled Latent Cognitive Dynamics

1. Core Objective
=================

The objective is to investigate whether machine reasoning can be improved by explicitly
controlling transitions between latent states rather than relying only on next-token
prediction or fixed reasoning loops.

The central architectural claim is:

Machine reasoning can be modeled and controlled as a trajectory through latent state
space, where reusable state-dependent operators transform the current state and a
higher-level controller selects among them in order to maintain productive dynamics.
```

The architecture does not assume that a latent state is equivalent to a complete thought. It treats latent state as an operational representation that can be observed, compared, transformed, and evaluated.

Terminology note:
Cognitive vocabulary (cognitive state, creative tension, restructuring) is used in motivational sections only. Method and experiment sections use operational terms (latent state, trajectory regime metric Ω, persistent reorganization Ψ) to avoid importing untested psychological claims into the technical definitions.

2. Scope

CLCD focuses on the following research questions:

1. Can latent states be operationally extracted from a language model?

2. Do different operator instructions produce statistically distinguishable latent transition patterns?

3. Can reusable operators be learned from these patterns?

4. Can those operators causally influence downstream reasoning?

5. Can a controller select operators dynamically according to the current trajectory?

6. Does maintaining a productive latent dynamic improve hypothesis generation, reasoning quality, or problem solving under equal compute budgets?

Out of scope for v0.2:

- modeling all aspects of human cognition
- claiming consciousness or subjective experience
- defining a universal theory of intelligence
- training a foundation model from scratch
- proving that latent states correspond one-to-one with semantic concepts
- unifying internal and external dialogue (deferred; see Section 27)

3. Related Work and Novelty Differentiation

CLCD composes mechanisms that individually exist in prior work. The spec must be evaluated against that work explicitly.

3.1 Activation steering / representation engineering

(steering vectors: Subramani et al. 2022 [1], Turner et al. 2023 [2];
contrastive activation addition: Rimsky et al. 2024 [3];
representation engineering: Zou et al. 2023 [4];
function vectors: Todd et al. 2024 [5];
in-context vectors: Liu et al. 2024 [6])

What exists:
Fixed vectors added to the residual stream reliably shift model behavior along interpretable directions; function vectors capture task identity.

What CLCD adds:
The intervention vectors are (a) organized into a small operator alphabet defined by reasoning function rather than by content or style, (b) allowed to be state-dependent rather than fixed, and (c) selected online by a controller reading trajectory metrics. The existence of steering effects is therefore treated as established prior art, not as a CLCD contribution; Experiment C tests whether steering-style intervention can reproduce *operator-specific reasoning functions*, which is the open question.

3.2 Structured reasoning search

(Tree of Thoughts: Yao et al. 2023 [7]; Graph of Thoughts: Besta et al. 2024 [8];
self-refine: Madaan et al. 2023 [9]; self-reflection: Shinn et al. 2023 [10];
multi-agent debate: Du et al. 2023 [11]; meta-prompting: Suzgun & Kalai 2024 [12];
Self-Discover: Zhou et al. 2024 [13])

What exists:
Search or scheduling over discrete reasoning moves expressed in text, with evaluators scoring intermediate text states.

What CLCD adds:
The state being tracked, scored, and controlled is a latent vector rather than text; the controller's inputs are geometric trajectory statistics (magnitude, directional diversity, coherence) rather than text-level self-evaluation; and at Stages 3+ the

operators act directly on activations rather than through prompts.

Consequence for interpretation:
At Stage 1 (prompt-implemented operators), CLCD is a member of this family, and any Stage-1 result must be reported as "adaptive prompt scheduling versus fixed prompt scheduling", not as evidence for latent control. See Section 4.3.

3.3 Latent-space reasoning

(continuous chain-of-thought / Coconut: Hao et al. 2024 [14]; recurrent hidden-state processing)

What exists:
Reasoning steps carried in continuous states without decoding to text.

What CLCD adds:
An explicit, discrete, human-auditable operator alphabet over those states, plus a controller and a trajectory-level objective, rather than end-to-end learned latent rollouts.

3.4 Novelty claim (restated)

The proposed novelty is not latent reasoning, activation steering, self-reflection, or multi-agent dialogue. It is the combination of:

1. an explicit operator alphabet defined by reasoning function
2. reusable state-dependent latent transformations implementing that alphabet
3. a higher-level controller selecting operators from trajectory geometry
4. optimization of trajectory dynamics rather than only final output
5. a measurable, pre-registered productive-regime hypothesis (Section 12)

The single most novel component is (3)-(4): closed-loop control over latent trajectory statistics. If ablations show the controller contributes nothing beyond a random operator schedule, the novelty claim collapses to prior art.

3.5 References

[1] Subramani, N., Suresh, N., Peters, M. (2022). Extracting Latent Steering Vectors from Pretrained Language Models. Findings of ACL 2022.
[2] Turner, A., Thiergart, L., Udell, D., Leech, G., Mini, U., MacDiarmid, M. (2023). Activation Addition: Steering Language Models Without Optimization. arXiv:2308.10248.
[3] Rimsky, N., Gabrieli, N., Schulz, J., Tong, M., Hubinger, E., Turner, A. (2024). Steering Llama 2 via Contrastive Activation Addition. ACL 2024.
[4] Zou, A., et al. (2023). Representation Engineering: A Top-Down Approach to AI Transparency. arXiv:2310.01405.
[5] Todd, E., Li, M., Sharma, A., Mueller, A., Wallace, B., Bau, D. (2024). Function Vectors in Large Language Models. ICLR 2024.
[6] Liu, S., Ye, H., Xing, L., Zou, J. (2023). In-Context Vectors: Making In-Context Learning More Effective and Controllable Through Latent Space Steering. arXiv:2311.06668.
[7] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., Narasimhan, K. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. NeurIPS 2023.
[8] Besta, M., et al. (2024). Graph of Thoughts: Solving Elaborate Problems with Large Language Models. AAAI 2024.
[9] Madaan, A., et al. (2023). Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS 2023.
[10] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 2023.
[11] Du, Y., Li, S., Torralba, A., Tenenbaum, J., Mordatch, I. (2024). Improving Factuality and Reasoning in Language Models through Multiagent Debate. ICML 2024.
[12] Suzgun, M., Kalai, A. (2024). Meta-Prompting: Enhancing Language Models with Task-Agnostic Scaffolding. arXiv:2401.12954.
[13] Zhou, P., et al. (2024). Self-Discover: Large Language Models Self-Compose Reasoning Structures. NeurIPS 2024.
[14] Hao, S., Sukhbaatar, S., Su, D., Li, X., Hu, Z., Weston, J., Tian, Y. (2024). Training Large Language Models to Reason in a Continuous Latent Space (Coconut). arXiv:2412.06769.

Citation details (venues, years) should be re-verified against the published versions before external circulation.

4. Fundamental Objects

4.1 Latent State

Let:

$s_t \in \mathbb{R}^d$

where s_t is the operational latent state at step t.

The state is extracted from model activations:

$s_t = P(H_t)$

where:

H_t:
hidden activations generated during the current reasoning episode

P:
a projection or pooling function

Possible implementations of P include:

- last-token hidden state
- mean pooling across selected tokens
- weighted attention pooling
- layer aggregation
- learned linear projection
- autoencoder projection
- state-summary embedding for early prototypes

The implementation of P must be fixed within an experiment.

Different implementations of P are considered different state encoders and must be evaluated separately.

4.2 Geometry Requirements

Raw transformer activations are anisotropic: a small number of outlier dimensions can dominate norms and cosines, and activation norms grow with depth and context length. Therefore:

Requirement G1:
All trajectory metrics (Sections 8, 11, 12) are computed in a normalized space, not on raw activations.

$s'_t = W (s_t - \mu)$

where μ and W are estimated on a held-out calibration set of episodes from the same model, layer, and encoder configuration. Acceptable choices for W:

- whitening (ZCA or PCA) to identity covariance
- per-dimension standardization as a minimum fallback
- a learned metric, treated as part of the state encoder

Requirement G2:
μ and W are frozen before any experiment that uses them and are logged with the run (Section 24). Recomputing the normalization mid-experiment invalidates the run.

Requirement G3:
Robustness check: key results (separability, Ω-outcome association) must be reported under at least two normalization choices to show they are not artifacts of one metric.

All formulas below assume the normalized state; primes are dropped for readability.

4.3 Causal Status of the State by Stage

The causal role of s_t differs across implementation stages (Section 22), and every result must be labeled with the status under which it was obtained.

Observational status (Stages 1-2):
Generation is driven by the full token context / KV cache. s_t is a lossy summary computed *after the fact* and does not causally drive generation. Consequences:

- Separability results (Experiment A) at these stages may reflect surface features of operator prompts propagated into activations, not functional differences.

  This is a measurement of representational footprint, not of control.

- Controller results (Experiment D) at Stage 1 demonstrate only that adaptive
  prompt scheduling can beat fixed scheduling. They must be reported as such.

Causal status (Stages 3+):
Operators modify activations directly through the injection mechanism (Section 6),
so s_t (or its pre-image in the residual stream) is on the causal path to the
output. Only results obtained under causal status can support the central claim
of latent control.

Hypotheses H1-H2 are testable under observational status. H3-H6 require causal
status to carry their intended meaning; Stage-1 versions of H4-H6 are treated as
pilot results only.

4.4 Transition Vector

The elementary observed transition is:

δ_t = s_(t+1) - s_t

Its magnitude is:

m_t = ||δ_t||

Its normalized direction is:

u_t = δ_t / ||δ_t||

Note: the difference form assumes local linear structure in the normalized space.
This is a modeling choice, not a finding; if transition statistics prove unstable,
alternative transition representations (e.g., log-map on a learned manifold) are a
fallback, treated as a different encoder under Section 4.1 rules.

The transition vector is not assumed to be intrinsically meaningful.

Its meaning must be established experimentally through:

- repeatability
- separability
- transferability
- intervention
- downstream effect

4.5 Episode

An episode is:

E = (x, s_0, a_0, s_1, a_1, ..., s_T, y)

where:

x:
problem, evidence, or task input

s_0:
initial latent state

a_t:
operator selected at step t

s_t:
latent state

y:
final decoded output

T denotes episode length in operator steps. (The symbol T with a subscript, T_a,
always denotes an operator; bare T denotes episode length.)

4.6 Operator

An operator is a transformation:

T_a : S -> S

where a identifies the intended reasoning function.

General form:

s_(t+1) = T_a(s_t, x, ξ_t)

where:

x:
active task context

ξ_t:
stochastic component

An operator is not merely a textual instruction.

It is considered validated only if it produces a repeatable and causally useful
transformation across multiple contexts.

5. Minimal Operator Alphabet

The initial alphabet contains four operators.

```
A = {
    EXPAND,
    CHALLENGE,
    CONNECT,
    CONSOLIDATE
}
```

This alphabet is intentionally minimal.

5.1 EXPAND

Purpose:

Increase search breadth.

Expected effects:

- generate alternative explanations
- introduce additional possibilities
- increase local diversity
- reduce premature convergence

Textual prototype:

"Generate multiple plausible alternatives that have not yet been considered."

5.2 CHALLENGE

Purpose:

Destabilize unsupported structure.

Expected effects:

- identify hidden assumptions
- search for contradictions
- generate counterexamples
- revise overconfident claims

Textual prototype:

"Identify and challenge the weakest unsupported assumption."

5.3 CONNECT

Purpose:

Create relational structure between separated ideas.

Expected effects:

- discover analogies
- bridge distant concepts
- identify common mechanisms
- produce integrative explanations

Textual prototype:

"Find a non-obvious but defensible connection between the current ideas."

5.4 CONSOLIDATE

Purpose:

Restore coherence and compress the current state.

Expected effects:

- remove redundancy
- select stronger explanations
- summarize current structure
- stabilize a candidate hypothesis

Textual prototype:

"Produce the simplest coherent model that preserves the strongest insights."

Prompt-artifact control:
Each operator requires at least five paraphrases of its textual prototype (raised from three in v0.1), plus at least two adversarial control prompts that share surface form with an operator but request a different function. Paraphrase identity is logged and used as a nuisance factor in the separability analysis (Section 17).

6. State Injection Mechanism
==========

v0.1 defined extraction (P) but not how a transformed state re-enters the model. Every intervention experiment (Stage 3+) depends on this definition.

6.1 Definition

An injection mechanism is a function:

I : (H_t, Δ) -> H'_t

that maps current activations and an intended state change Δ to modified activations, such that generation proceeds from H'_t.

CLCD v0.2 fixes the following default implementation:

Residual-stream addition.

An operator effect Δ_norm is estimated in the *normalized* state space (Section 4.2), while residual-stream activations h live in the *raw* activation space. The mapping back therefore has two explicit steps:

Step 1 — de-normalize (normalized state space -> raw state space):

$\Delta_{raw} = W^{-1} \Delta_{norm}$

The centering term μ cancels because Δ is a difference of states, not a state. If W is a non-square or rank-deficient whitening matrix, its pseudoinverse is used and logged.

Step 2 — lift to the residual stream and add:

$h'_{(l, i)} = h_{(l, i)} + \alpha * P^{+}(\Delta_{raw})$

where:

h_(l, i):
residual-stream activation at layer l, token position i (raw space)

P^{+}:
the (pseudo-)inverse of the state encoder's projection, mapping raw state space to the residual stream; for mean pooling over a token set this is broadcast addition over that set; for a learned linear P it is the Moore-Penrose pseudoinverse; for nonlinear encoders a learned decoder must be trained and

validated for reconstruction error before use

α:
intervention strength, a logged hyperparameter

Consistency rule: any Δ reported, compared, or learned (e.g., δ_a in Experiment C) is expressed in normalized space; conversion to raw space happens only inside the injection mechanism, in the two steps above. The reconstruction sanity check (6.3) is likewise evaluated in normalized space.

6.2 Required intervention configuration

Every intervention run must specify and log (Section 24):

- injection layer(s) l (default: the same layer(s) used for extraction)
- token positions i (choices: last token only; all generated tokens; a fixed window)
- schedule: single-shot (once at operator application) versus persistent (re-applied at every generation step until the next operator)
- strength α and any norm-matching or clamping applied
- whether attention KV entries are also modified (default: no)

Default configuration for the first intervention experiments:
same layer as extraction, all subsequently generated token positions, persistent schedule, α selected on a calibration split by matching the norm of naturally observed δ for the same operator.

6.3 Validity checks

Before behavioral evaluation, each injection configuration must pass:

- Reconstruction sanity: extracting the state immediately after injection recovers $s_t + \Delta$ within a tolerance ε (logged).
- Fluency preservation: perplexity of continuations under injection must not degrade beyond a pre-set bound relative to no-injection controls; interventions that break generation are classified as out-of-distribution, not as evidence about operators.

6.4 Failure semantics

If no configuration passes 6.3, the correct conclusion is that this state encoder / injection pair is inadequate — not that operator geometry is absent. This distinction feeds Decision Gate 3 (Section 25).

7. Transition Generator
==========

The transition generator is:

F_θ

with:

$s_{t+1} = F_\theta(s_t, a_t, x, \xi_t)$

Possible implementations:

Stage 1:
Prompt-conditioned LLM generation followed by state re-encoding
(observational status; see 4.3)

Stage 2:
Prompt-conditioned hidden-state observation
(observational status)

Stage 3:
Direct latent intervention via the injection mechanism of Section 6
(causal status)

Stage 4:
Learned operator modules
(causal status)

Stage 5:
Jointly trained transition generator
(causal status)

The generator may produce multiple candidate next states:

{s~_(t+1)^1, ..., s~_(t+1)^m}
    = GenerateTransitions(s_t, a_t, x)

A selector may then choose:

s_(t+1)
    = argmax_s J_state(s | s_t, a_t, x)

8. Measurement Definitions

v0.1 used coherence, hypothesis quality, hypothesis potential, and testability as primitives without operational definitions, while feeding them into the controller, the evaluator, and Ω. This created two problems: unmeasurable constructs, and a circularity risk when the same judge drives both inner-loop selection and outcome evaluation. This section fixes both.

8.1 Metric registry

Every quantity used by any module must be one of:

Type G (geometric):
computed from normalized states and transitions only.
Examples: m_t, u_t, M_t, V_t, ρ_t, Ψ_t.

Type E (embedding-based):
computed from cosine similarities between state embeddings and reference embeddings (e.g., the encoded problem statement).
Example: coherence C_t (8.2).

Type J (judge-based):
produced by an LLM judge or human rater from decoded text.
Examples: Q_t, H(s), U(s).

Type O (objective):
computed from ground truth on hidden-solution tasks.
Example: final task score.

8.2 Coherence C_t (Type E)

C_t = cos(s_t , e_x)

where e_x is the encoding of the active problem statement under the same state encoder and normalization. Optional variant: mean cosine to the problem statement and to the current best candidate hypothesis state. The variant used is fixed per experiment and logged.

8.3 Repetition ρ_t (Type G)

ρ_t = max over j in [t-k, t-1] of cos(s_t, s_j)

High ρ_t indicates return to previously visited regions.
(v0.1 called this R_t; renamed to avoid collision with restructuring.)

8.4 Candidate hypothesis quality Q_t, hypothesis potential H(s), testability U(s) (Type J)

These are judge-based scores over decoded text on fixed rubrics (0-10 scales):

Q_t: internal consistency, evidence contact, and specificity of the current candidate hypothesis
H(s): whether the decoded state contains material from which a hypothesis could be formed
U(s): whether the decoded content implies observable predictions
(v0.1 called this T(s); renamed to avoid collision with operators T_a.)

8.5 Judge separation protocol

Rule J1:
The judge model (or rubric instance) used inside the loop — by the state evaluator or the controller — must be different from the judge used for outcome evaluation (different model family, or human raters for outcomes).

```
Rule J2:
Primary conclusions for H4-H6 must rest on Type O metrics (hidden-solution tasks)
wherever available. Type J outcome metrics are secondary and must be reported with
inter-rater or inter-judge agreement.

Rule J3:
Any experiment in which inner-loop selection and outcome evaluation share a judge
must be labeled as such and cannot support a headline claim.

Rationale: if the controller optimizes the same score that defines success, gains
are guaranteed by construction and demonstrate selection, not improved reasoning.

9. State Evaluator
==================

The state evaluator estimates the local utility of a candidate state.

J_state(s)
    = w_r R(s)
    + w_n N(s)
    + w_c C(s)
    + w_h H(s)
    + w_u U(s)
    - w_d D(s)

where:

R(s):
relevance to the active problem (Type E: cosine to problem encoding)

N(s):
novelty relative to prior states (Type G: 1 - max cosine to trajectory history)

C(s):
coherence (Type E, Section 8.2)

H(s):
hypothesis potential (Type J, Section 8.4)

U(s):
testability potential (Type J, Section 8.4)

D(s):
drift or redundancy penalty (Type G)

The weights are experimental parameters and are subject to the tuning-budget rules
of Section 20. Type J components inside J_state fall under judge Rule J1.

10. Controller
==============

The controller selects the next operator:

a_t = Π_φ(z_t)

where z_t summarizes the current trajectory.

Minimum controller state:

z_t = (
    M_t,
    V_t,
    C_t,
    ρ_t,
    Q_t,
    A_hist_t
)

where:

M_t:
average transition magnitude over the recent window

V_t:
directional diversity

C_t:
coherence (Section 8.2)
```

ρ_t:
repetition / stagnation (Section 8.3)

Q_t:
quality of current candidate hypothesis (Section 8.4)

A_hist_t:
the sequence (or counts) of recently applied operators
(added in v0.2; required by the controller interface in Section 14.3 and by
Rules 2 and 5 below, which reference recent operator usage)

10.1 Rule-Based Controller v0.2

The rule set below contains 8 thresholds. All thresholds are hyperparameters and
fall under the equal-tuning-budget protocol of Section 20.

Rule 1: Stagnation

If:

M_t < θ_low

or:

ρ_t > θ_repeat

then select:

EXPAND or CONNECT

Rule 2: Premature agreement

If:

C_t > θ_coherence_high

and A_hist_t contains no CHALLENGE within the last k steps,

then select:

CHALLENGE

Rule 3: Instability

If:

M_t > θ_high

and:

V_t > θ_diversity_high

and:

C_t < θ_coherence_low

then select:

CONSOLIDATE

Rule 4: Fragmentation

If multiple stable but weakly connected regions are detected
(operationalized: clustering of the last k states yields ≥ 2 clusters with
inter-cluster cosine below θ_frag),

then select:

CONNECT

Rule 5: Candidate maturity

If:

Q_t > θ_quality

and A_hist_t contains no CHALLENGE since the last consolidation,

then select:

CHALLENGE

Rule 6: Stopping

Stop when one of the following is satisfied:

- hypothesis quality exceeds threshold
- testability exceeds threshold
- no operator produces improvement
- maximum compute budget is reached
- trajectory collapses into repetition (ρ_t persistently above θ_repeat)

Stopping-rule note: different stopping behavior changes realized compute. Compute matching (Section 17.4) is applied to *realized* compute, not nominal budgets.

11. Trajectory Regime Metric Ω

(Motivational name: Creative Tension. Method sections use "trajectory regime metric".)

Ω_t is a macroscopic property of the recent latent trajectory, computed over a window of k steps from normalized quantities.

Components (each rescaled to [0, 1] using calibration-set statistics, so the product is not dominated by scale differences):

M̂_t:
mean transition magnitude over the window, rescaled

$$M_t = (1/k) \Sigma ||\delta_i||$$

V_t:
directional diversity (already in [0, 1])

$$V_t = 1 - ||(1/k) \Sigma u_i||$$

C_t:
coherence (Section 8.2), mapped from [-1, 1] to [0, 1]

11.1 Target-Zone Form

The architecture assumes that productive dynamics may occur inside an intermediate region rather than at maximum movement or maximum diversity.

Define:

$$g(\hat{M}_t) = \exp(-(\hat{M}_t - \mu_M)^2 / (2\sigma_M^2))$$

$$h(V_t) = \exp(-(V_t - \mu_V)^2 / (2\sigma_V^2))$$

Then:

$$\Omega_t = g(\hat{M}_t) * h(V_t) * C_t$$

Interpretation:

- very low movement: stagnation
- very high movement with low coherence: drift
- moderate structured movement: potentially productive regime

The multiplicative form is an initial modeling choice, not a claim; an additive form is a required ablation (Section 21).

11.2 Pre-registration requirement

The parameters μ_M, σ_M, μ_V, σ_V and the H5 zone bounds [Ω_L, Ω_U] are free parameters. With free bounds, some correlating interval can almost always be found post hoc, which would make H5 unfalsifiable in practice. Therefore:

Rule Ω1:
All Ω parameters and zone bounds must be either (a) pre-registered before the

```
confirmatory run, based on pilot data, or (b) selected by cross-validation:
fitted on a development split and evaluated once on a held-out confirmatory split.

Rule Ω2:
Exploratory scans over bounds are permitted but must be reported as exploratory,
with the number of configurations tried.

Rule Ω3:
H5 is evaluated only on the held-out split, with the multiple-comparison
corrections of Section 17.

12. Restructuring Ψ_t
=====================

Restructuring is persistent reorganization rather than simple accumulation.
(v0.1 symbol R_t; renamed to Ψ_t to avoid collision with repetition.)

Initial approximation:

Ψ_t
    = ||s_after - s_before||
      * Persistence_t
      * Coherence_t

Persistence may be estimated as:

Persistence_t
    = (1/k) Σ cos(
        s_(t+j) - s_(t-1),
        s_t - s_(t-1)
      )

A restructuring event is considered stronger when:

- the state changes materially
- the new direction persists
- the new state remains relevant
- future predictions or hypotheses change

13. System Architecture
=======================

Logical pipeline:

Problem / Evidence
        |
        v
State Encoder  (P, normalization W, μ)
        |
        v
Latent State s_t
        |
        v
Trajectory Analyzer
        |
        v
Trajectory Regime Metric Ω_t
        |
        v
Controller Π
        |
        v
Selected Operator a_t
        |
        v
Transition Generator F
  (Stage 1-2: prompt path)
  (Stage 3+: injection mechanism I, Section 6)
        |
        v
Candidate Next States
        |
        v
State Evaluator
        |
        v
Selected State s_(t+1)
        |
        v
```

```
Stopping Condition
        |
        v
Decoder / Final Hypothesis
```

14. Module Interfaces

14.1 State Encoder

Input:

- model activations
- token positions
- selected layers
- pooling configuration
- normalization parameters (μ, W) and calibration-set identifier

Output:

- normalized latent state s_t
- encoder metadata
- confidence or reconstruction error

14.2 Trajectory Analyzer

Input:

- recent states
- recent transition vectors
- task representation

Output:

- M_t
- V_t
- C_t
- ρ_t (repetition score)
- fragmentation score
- Ω_t

14.3 Controller

Input:

- trajectory metrics
- operator history A_hist_t
- compute budget consumed so far
- candidate hypothesis state

Output:

- selected operator
- operator confidence
- decision rationale for logging

14.4 Transition Generator

Input:

- current latent state
- selected operator
- task context
- stochastic seed
- (Stage 3+) injection configuration (Section 6.2)

Output:

- one or more candidate next states
- optional decoded intermediate reasoning

14.5 State Evaluator

Input:

```
- candidate next states
- current state
- task context
- prior trajectory

Output:

- candidate scores with metric-type labels (Section 8.1)
- selected next state
- score decomposition

14.6 Decoder
------------

Input:

- final latent state
- task context

Output:

- final hypothesis
- reasoning summary
- proposed tests
- confidence estimates

15. Training Objectives
=======================

The architecture may eventually require several objectives.

15.1 Operator Separability
--------------------------

Encourage transition distributions to be distinguishable:

L_sep
    = classification loss over operator labels from δ_t

15.2 Operator Consistency
-------------------------

Encourage similar operators to produce comparable functional effects across contexts:

L_consistency
    = distance between normalized transition distributions

15.3 Task Utility
-----------------

L_task
    = downstream reasoning or solution loss

15.4 Trajectory Quality
-----------------------

L_traj
    = penalty for stagnation, drift, incoherence, and unnecessary length

15.5 Intervention Fidelity
--------------------------

Ensure that direct latent intervention produces the intended operator behavior:

L_intervention
    = behavioral mismatch after operator application

15.6 Combined Objective
-----------------------

L_total
    = λ_1 L_task
    + λ_2 L_sep
    + λ_3 L_consistency
    + λ_4 L_traj
    + λ_5 L_intervention
```

16. Operator Discovery Experiment

16.1 Objective

Test whether EXPAND, CHALLENGE, CONNECT, and CONSOLIDATE correspond to statistically distinguishable and causally useful latent transformations.

16.2 Dataset

Construct reasoning episodes from multiple domains:

- causal puzzles
- scientific explanation tasks
- abductive reasoning
- design problems
- synthetic hypothesis reconstruction
- contradiction resolution

Sizing (revised from v0.1; see Section 17.2 for rationale):

- Pilot (Stage 1): ≥ 30 problems per domain across ≥ 3 domains, each problem run under all 4 operators and ≥ 5 paraphrases per operator, ≥ 3 seeds. This yields ≥ 5,400 labeled transitions for the pilot classifier.
- Confirmatory: ≥ 50 problems per domain across ≥ 4 domains, with a full leave-one-domain-out protocol.

Each episode must include:

- initial problem
- pre-operator state
- explicit operator instruction (with paraphrase ID)
- post-operator state
- final output
- task outcome

16.3 Data Collection

For every problem and operator:

1. Encode pre-operator state:

   s_before

2. Apply textual prototype of operator a (paraphrase sampled and logged).

3. Encode post-operator state:

   s_after

4. Compute:

   δ_a = s_after - s_before

5. Store:

   (domain, problem, operator, paraphrase_id, seed,
    s_before, s_after, δ_a, output, score)

16.4 Experiment A: Separability

Train a classifier:

a_hat = f_cls(δ)

Measure:

- accuracy
- macro F1
- confusion matrix
- cross-domain performance (leave-one-domain-out)
- cross-paraphrase performance (leave-paraphrases-out): train on a subset of paraphrases, test on held-out paraphrases; separability that survives this split is less likely to be a prompt-wording artifact

Success criterion:

Operator prediction must exceed matched baselines and remain above chance under both cross-domain and cross-paraphrase testing, at the effect-size and significance thresholds of Section 17.

Status note: under Stages 1-2 this experiment has observational status (4.3) and measures representational footprint, not control.

16.5 Experiment B: Transfer

Estimate operator transformations on training domains.

Apply them to unseen domains.

Measure whether intended effects remain detectable.

16.6 Experiment C: Intervention

Apply operator transformation without textual operator instruction, using the injection mechanism and validity checks of Section 6.

Operator function classes, in order of increasing capacity:

Fixed vector:

$T_a(s) = s + v_a$

Linear operator:

$T_a(s) = A_a s + b_a$

State-dependent operator:

$T_a(s) = s + v_a(s)$

Nonlinear operator:

$T_a(s) = f_a(s)$

Measure whether the resulting behavior matches the intended function, using blind Type J raters (Rule J1) and Type O outcomes where available.

A negative result at one capacity class licenses moving to the next class, not rejection of H3; rejection requires failure across classes with passing injection-validity checks (Section 6.3).

16.7 Experiment D: Downstream Utility

Compare:

A. Standard Q&A

B. Fixed operator cycle

C. Dynamic rule-based controller

D. Learned controller

Control for:

- model
- temperature
- token budget
- number of reasoning steps
- sampling budget
- external tools
- hyperparameter tuning budget (Section 20)
- realized compute including controller overhead (Section 17.4)

Measure:

- solution quality (Type O primary)
- hypothesis quality (Type J, separated judge)
- testability
- novelty

- coherence
- compute efficiency

16.8 Experiment E: Mediation by Ω

Test whether controller performance is associated with time spent inside the pre-registered productive Ω region (Rule Ω1).

The claim is stronger if:

```
Controller
    -> changes Ω trajectory
    -> improves downstream result
```

rather than merely:

```
Controller
    -> uses more computation
    -> improves result
```

Analysis: formal mediation analysis with realized compute entered as a covariate; partial effects reported with confidence intervals.

17. Statistical Analysis and Compute Accounting
==========

(New in v0.2. v0.1 specified metrics but no inference procedure.)

17.1 Units, seeds, and runs

- The unit of analysis for downstream comparisons is the problem instance; transitions are nested within problems and domains.
- Every condition is run with ≥ 5 random seeds. Reported values are means over seeds with 95% confidence intervals.
- Nested structure (transitions within problems within domains) is handled with mixed-effects models or hierarchical bootstrap; naive pooling of transitions as independent observations is not accepted.

17.2 Power and sizing rationale

Cell counts under the v0.1 sizing (30-100 total tasks) were insufficient: 4 operators × 6 domains over 100 tasks yields ~4 transitions per cell. The Section 16.2 sizing targets ≥ 100 transitions per operator-domain cell in the pilot and supports leave-one-domain-out evaluation in the confirmatory phase. A pilot-based power analysis must precede the confirmatory run; the confirmatory sample size is fixed from that analysis and pre-registered.

17.3 Tests and multiple comparisons

- Separability (H1): permutation test against label-shuffled null; report macro F1 with permutation p-value and bootstrap CI.
- Utility comparisons (H4): pairwise tests on per-problem scores with Holm-Bonferroni correction across the condition pairs.
- The hypothesis family H1-H6 and the 10 falsification criteria constitute multiple comparisons; confirmatory claims use a family-wise correction, and each hypothesis is labeled confirmatory or exploratory in advance.
- Minimum effects of interest (pre-registered defaults, adjustable in the pre-registration only): separability macro F1 ≥ 0.45 over a 0.25 chance level (4 classes); downstream utility gain ≥ 0.3 standardized effect (Cohen's d) over the strongest matched baseline.

17.4 Definition of equal compute

"Equal compute" in H4 and Experiment D means equal *realized* cost per problem, accounted in two ledgers, both logged:

Ledger 1 — model tokens: all tokens generated and consumed by the base model, including operator prompts, decoded intermediate reasoning, and judge calls made inside the loop.

Ledger 2 — total FLOPs (or wall-clock proxy): Ledger 1 plus state extraction, normalization, trajectory analysis, controller execution, and candidate-state evaluation.

A controller advantage that exists under Ledger 1 but disappears under Ledger 2 must be reported as an efficiency trade-off, not as a reasoning improvement. Because stopping rules change realized compute (Section 10.1), matching is enforced on realized, not nominal, budgets — e.g., by granting baselines the same realized-compute distribution via extended sampling.

18. Hypotheses

H1: Operator Separability (observational status sufficient)

Transition vectors conditioned on different operators are statistically distinguishable:

$P(\hat{a} = a \mid \delta_a) > \text{chance}$

under cross-domain and cross-paraphrase evaluation (16.4).

H2: Cross-Domain Transfer (observational status sufficient)

Operator transformations learned in one domain preserve measurable functional effects in unseen domains.

H3: Causal Intervention (requires causal status)

Direct application of a learned operator transformation, via the injection mechanism of Section 6, changes reasoning behavior in the intended direction without an explicit textual instruction.

H4: Dynamic Controller Advantage (stage-qualified)

Under equal realized compute (17.4):

$\text{Quality}_{\text{dynamic}} > \text{Quality}_{\text{fixed}} > \text{Quality}_{\text{QA}}$

H4-S1 (Stage 1 form): adaptive prompt scheduling outperforms fixed prompt scheduling. This is a pilot claim about scheduling, not about latent control.

H4-S3 (Stage 3+ form): the advantage persists when operators are applied by latent intervention. Only H4-S3 supports the central claim.

H5: Productive Regime Zone (requires pre-registration, Rule Ω1)

High-quality hypotheses occur more frequently when:

$\Omega_t \in [\Omega_L, \Omega_U]$

than outside it, where $[\Omega_L, \Omega_U]$ was fixed before the confirmatory run.

H6: Mediation

The benefit of the controller is partly mediated by changes in latent trajectory dynamics rather than only by additional tokens or steps, with realized compute as a covariate (16.8).

19. Falsification Criteria

CLCD v0.2 is weakened or falsified in its simple form if:

1. Operator-conditioned transition distributions cannot be distinguished beyond prompt artifacts (i.e., separability fails the cross-paraphrase split).

2. Separability disappears under cross-domain evaluation.

3. Learned transformations fail to cause the intended behavior across all capacity classes of 16.6, with passing injection-validity checks.

4. Direct intervention degrades reasoning or produces arbitrary effects despite passing the fluency-preservation check.

5. Dynamic policies do not outperform matched baselines under equal tuning budgets (Section 20).

6. Any performance gain disappears after equalizing realized compute (Ledger 2, 17.4).

7. Ω_t does not predict outcome quality on the held-out confirmatory split.

8. A simpler metric fully explains all observed gains.

9. Operators fail to transfer between contexts.

10. Latent state extraction is too unstable to support reproducible measurement
    (i.e., Requirement G3 robustness checks fail).

A negative result may falsify only a specific implementation, such as fixed-vector operators or a particular injection configuration (6.4), without falsifying the broader controlled-dynamics hypothesis. Such scoping must be declared in advance per Decision Gate, not constructed after the result.

20. Baselines and Tuning-Budget Protocol

Required baselines:

- direct answer
- chain-of-thought
- self-reflection
- fixed deliberation loop
- majority sampling
- tree search
- multi-agent debate
- prompt-only operator cycle
- equal-token random operator policy

The random operator policy is especially important because it tests whether operator *selection* matters beyond extra computation.

Tuning-budget protocol (new in v0.2):

The rule-based controller has ≥ 8 free thresholds (10.1) and the evaluator has free weights (Section 9). A tuned controller compared against untuned baselines produces artifactual gains. Therefore:

- Every condition (baselines included) receives the same hyperparameter search budget, defined as an equal number of configuration evaluations on the same development split.
- Baselines with no natural hyperparameters spend the budget on their available knobs (temperature, exemplar selection, sampling width, tree depth).
- The search space, budget, and selected configurations are logged for every condition.
- Final comparisons run once on the held-out confirmatory split.

21. Ablation Studies

Remove or modify one component at a time:

- no trajectory regime estimator (Ω)
- additive instead of multiplicative Ω (11.1)
- no controller
- fixed operator order
- random operator order
- three operators instead of four
- no CONNECT
- no CHALLENGE
- no state evaluator
- fixed-vector versus state-dependent operators
- single-layer versus multi-layer state encoder
- no persistence metric
- alternative normalization (Requirement G3)

The architecture is supported only if its key components contribute independently.

22. Implementation Stages

Each stage is labeled with the causal status of its results (Section 4.3).

Stage 1: Symbolic Orchestration [observational]

Operators are implemented through prompts.

Goal:

Validate task design, logging, baselines, and controller logic.

Interpretation constraint: results at this stage are adaptive-prompt-scheduling
results (Section 3.2) and support H4-S1 only.

Stage 2: Latent Observation [observational]

Extract hidden states and measure transitions.

Goal:

Test separability (H1) and trajectory metrics.

Stage 3: Latent Intervention [causal]

Apply activation-level transformations via the injection mechanism (Section 6).

Goal:

Test causality (H3).

Stage 4: Learned Operators [causal]

Learn T_a for each function class (16.6).

Goal:

Replace textual prompts with reusable latent transformations.

Stage 5: Dynamic Controller [causal]

Learn Π(a_t | z_t).

Goal:

Select operators according to trajectory dynamics (H4-S3).

Stage 6: Joint Training [causal]

Jointly optimize:

- state encoder
- operators
- controller
- state evaluator
- stopping policy

Stage 7: Emergent Alphabet [causal]

Allow the architecture to discover additional operators or merge redundant ones.

Goal:

Determine whether the original human-defined alphabet is necessary.

23. Research Risks
==

23.1 Representation Artifact

Observed operator differences may reflect prompt wording rather than function.

Mitigation:

- ≥ 5 paraphrases per operator plus adversarial controls (Section 5)
- cross-paraphrase splits in Experiment A (16.4)
- cross-domain validation
- prompt-free latent intervention (Stage 3)

23.2 Layer Dependence

Operator geometry may exist only in specific layers.

Mitigation:

- multi-layer analysis
- layer-wise probing
- projection robustness tests

23.3 Model Dependence

Operators may not transfer across model families.

Mitigation:

- treat cross-model transfer as a separate hypothesis
- begin with within-model validation

23.4 Semantic Overclaim

Latent vectors may correlate with behavior without representing interpretable thoughts.

Mitigation:

- operational terminology in method sections (Section 1)
- require causal intervention
- avoid claims of semantic identity

23.5 Compute Confound

Improvement may come only from extra reasoning steps.

Mitigation:

- two-ledger realized-compute matching (17.4)
- token-budget controls
- random-policy baselines

23.6 Evaluation Bias and Circularity

LLM judges may favor familiar structure; a shared judge between inner loop and outcome evaluation guarantees artifactual gains.

Mitigation:

- judge separation protocol (8.5)
- objective hidden-solution tasks as primary outcomes
- human blind review
- downstream testing

23.7 Metric Instability

Trajectory geometry may be an artifact of a particular normalization.

Mitigation:

- Requirements G1-G3 (Section 4.2)
- report key results under ≥ 2 normalization choices

24. Logging Requirements
========================

Every run should record:

- model and checkpoint
- layer selection
- state encoder and normalization parameters (μ, W, calibration-set ID)
- random seed
- problem identifier and domain
- operator selected and paraphrase ID
- operator source (controller rule fired, or policy output)
- injection configuration when applicable (Section 6.2) and validity-check results
- state before / state after
- transition vector
- trajectory metrics (M_t, V_t, C_t, ρ_t, Ψ_t)
- Ω_t and its parameters
- candidate state scores with metric-type labels (8.1)
- judge identity for every Type J score
- final output
- final task score (Type O where available)
- realized compute, both ledgers (17.4)
- hyperparameter search budget consumed (Section 20)

- stopping reason

Reproducibility is a design requirement.

25. Decision Gates

Gate 1:
Are operator transitions separable under cross-paraphrase and cross-domain splits?

If no:
Revise state encoder or operator definitions.

Gate 2:
Do operator effects transfer across contexts?

If no:
Test state-dependent or nonlinear operators.

Gate 3:
Can direct intervention reproduce operator behavior?

If no, and injection-validity checks (6.3) pass:
The observed geometry may be correlational only.

If no, and injection-validity checks fail:
The encoder / injection pair is inadequate (6.4); revise it before concluding anything about operator geometry.

Gate 4:
Does the controller improve performance under equal realized compute and equal tuning budget?

If no:
Revise policy or reject the controller hypothesis.

Gate 5:
Does Ω_t explain useful variance on the held-out confirmatory split?

If no:
Replace or remove the trajectory regime construct.

26. Minimum Prototype

The minimum prototype may be implemented without training a new model.

Components:

1. Open-weight LLM with hidden-state access

2. State encoder using one selected layer, with normalization (4.2)

3. Four textual operator prompts with ≥ 5 paraphrases each

4. Rule-based controller

5. Logging of s_t and δ_t per Section 24

6. Basic trajectory metrics

7. Hidden-solution reasoning dataset sized per Section 16.2 (pilot tier)

8. Three baselines:
   - direct answer
   - fixed cycle
   - random operator policy at equal realized compute

The prototype is successful if it produces usable experimental data, even if it does not improve task performance.

27. Future Work (deferred from v0.1)

Internal and external dialogue as a unified transition source.

v0.1 proposed treating internal dialogue (source(a_t) = same entity) and external dialogue (source(a_t) = different entity) under one transition equation. The idea is retained as a research direction but removed from the core specification: it

added no testable content to v0.2 experiments and was underdeveloped relative to the rest of the document. It should return only with its own operationalization and experiment design.

Cross-model operator transfer (23.3) is likewise future work.

28. Current Central Claim

Machine reasoning may be treated as a controllable latent dynamical process.

Reusable operators transform the current latent state.

A higher-level controller selects among those operators according to the recent trajectory.

The system seeks neither maximum stability nor maximum disruption. It seeks a productive region in which coherent restructuring and useful hypothesis generation become more likely.

Scope note: this claim is supported only by results obtained under causal status (Stages 3+). Observational-stage results inform feasibility but cannot establish it.

29. Immediate Next Actions

1. Select one open-weight model with hidden-state access.

2. Select one layer or define a simple layer-combination method; fit and freeze normalization on a calibration split (4.2).

3. Build the pilot dataset per Section 16.2 (≥ 30 problems per domain, ≥ 3 domains, hidden solutions).

4. Write ≥ 5 paraphrases and ≥ 2 adversarial controls per operator.

5. Implement state extraction and transition logging (Section 24).

6. Run Experiment A with cross-paraphrase and cross-domain splits.

7. Run a pilot power analysis; pre-register confirmatory sample sizes, Ω parameters, and minimum effect sizes (17.2, 11.2).

8. Compare fixed-cycle, random-policy, and rule-based controller at equal realized compute and equal tuning budget.

9. Verify the Section 3.5 citations against published versions before external circulation.

10. Publish negative as well as positive results.

11. Revise this specification based on measured behavior.

12. Do not expand the theory until the first operator dataset exists.

End of CLCD Architecture Specification v0.2

Appendix B — CLCD Trajectory-First Research Plan v0.2 (verbatim)

```
CLCD Trajectory-First Research Plan v0.2
========================================

Controlled Latent Cognitive Dynamics

Status
------
Working research plan.

Purpose
-------
This document defines the trajectory-first development path for CLCD.

It does not replace the CLCD Architecture Specification v0.2 (hereafter "the
Spec"). It changes the order of investigation so that operators, trajectory
regimes, and controller policies are discovered from data rather than assumed
in advance.

The central methodological shift is:

Trajectories
    ->
Recurring Functional Modes
    ->
Causal Operators
    ->
Trajectory Regimes
    ->
Closed-Loop Controller

Version
-------
v0.2

Changes from v0.1 (summary)
---------------------------
1.  Added Section 3 (Binding Protocols): the Spec's metric registry, judge
    separation protocol, geometry requirements, statistical plan, and
    tuning-budget protocol are declared binding for this plan. v0.1 silently
    dropped them, reintroducing the circularity the Spec had fixed.
2.  Added Section 4 (Step Definition). What constitutes one latent reasoning
    step was an action item in v0.1; it is foundational, because every
    trajectory feature presupposes the time discretization. Step definitions
    are now treated like state encoders: fixed per experiment, with ≥ 2
    variants required for robustness.
3.  Trajectory features now have explicit discrete-space definitions
    (Section 5.5); v0.1 named velocity, acceleration, and curvature without
    defining them.
4.  Added seed-label independence criterion (Section 9.5): discovered modes
    must be shown not to merely recover the seed intervention labels
    (mutual-information check, cross-source recurrence).
5.  Added outcome-conditioning confound controls (Section 8.6): matched-length
    controls and difficulty stratification for successful-versus-unsuccessful
    comparisons.
6.  d_functional weight sensitivity requirement and an operational definition
    of d_context (Section 9.2-9.3).
7.  Defined how the desired regime r* is set, with data splits preventing
    leakage between regime discovery and controller training (Section 12.3).
8.  Added Section 13 (Additional Related Work): options / skill discovery in
    hierarchical RL and latent dynamics methods from computational
    neuroscience.
9.  Added a parallel low-cost track (Section 8.7): the Spec's Experiment A
    (seed-operator separability) runs on Atlas data at near-zero marginal
    cost, de-risking the slower discovery-first path.
10. Atlas sizing targets added (Section 8.2).
11. Symbol hygiene: the temporal window length is now w (v0.1 used k, which
    collided with the mode index k in O_k, T_k). Ω_vec dimensions are marked
    as named placeholders pending operational definitions (Section 11.4).
12. Gate A extended with failure semantics distinguishing "no structure" from
    "inadequate representation or step definition" (Section 15).

1. Research Motivation
======================

The Spec defines:
```

- latent states
- transition vectors
- candidate cognitive operators
- a trajectory regime metric
- a higher-level controller
- causal intervention stages

However, the architecture still assumes that human-defined operators such as EXPAND, CHALLENGE, CONNECT, and CONSOLIDATE are the correct functional primitives.

The trajectory-first plan removes that assumption.

The first scientific question becomes:

Does the space of latent reasoning trajectories contain stable, recurring, functionally meaningful, and causally reusable structure?

If no such structure exists, the operator-based architecture is weakened.

If such structure exists, operators should be derived from that structure rather than imposed as labels.

2. Revised Core Hypothesis

Let a reasoning trajectory be:

$\Gamma = (s_0, s_1, \ldots, s_T)$

where:

$s_t \in R^d$

is the normalized latent state at step t (normalization per Spec Section 4.2).

The local transition is:

$\delta_t = s_{t+1} - s_t$

The revised core hypothesis is:

There exist recurring trajectory motifs or transformation modes that:

1. appear across multiple problems and domains
2. produce similar downstream functional effects
3. can be represented by reusable transformations
4. causally influence future reasoning when applied to unseen states
5. can be selected by a controller to improve reasoning under matched compute

3. Binding Protocols Inherited from the Spec

(New in v0.2.)

The following Spec protocols apply to every phase of this plan without restatement. Any experiment violating them cannot support a headline claim.

B1 — Metric registry (Spec 8.1):
Every quantity is labeled Type G (geometric), Type E (embedding), Type J (judge), or Type O (objective). This matters doubly here, because functional effect profiles (Section 5.6) mix types.

B2 — Judge separation (Spec 8.5):
No judge used inside mode discovery or controller loops may also serve as an outcome judge. If modes are discovered using Type J effect components and later validated by the same judge, the discovery is circular by construction.

B3 — Geometry requirements (Spec 4.2, G1-G3):
All trajectory features are computed in normalized space; key results must be shown under ≥ 2 normalization choices.

B4 — Statistical plan (Spec 17):
Seeds (≥ 5), mixed-effects or hierarchical bootstrap for nested data, permutation nulls, family-wise correction, pre-registered minimum effects, confirmatory/exploratory labeling, and two-ledger realized-compute accounting. This plan adds Atlas-specific sizing in Section 8.2 but adds no exemptions.

B5 — Tuning budget (Spec 20):
Equal hyperparameter search budget across all compared conditions, including
clustering pipelines when cluster-based claims are compared against baselines.

B6 — Injection mechanism (Spec 6):
All causal interventions in Phase C use the Spec's two-step injection
(de-normalize, then lift) with its validity checks and failure semantics.

4. Step Definition

(New in v0.2; promoted from v0.1 Immediate Action 4.)

Every trajectory feature — velocity, acceleration, curvature, recurrence —
presupposes a time discretization. Different step definitions produce
different geometry, and Atlas results could be artifacts of this choice
alone. Therefore step definitions are governed like state encoders.

4.1 Candidate step definitions

ST-1 Token step:
      one state per generated token (finest; noisiest; longest sequences)

ST-2 Sentence / clause step:
      one state per sentence boundary in decoded reasoning

ST-3 Reasoning-move step:
      one state per prompted intervention or self-delimited reasoning move
      (coarsest; matches the Spec's operator step)

4.2 Rules

S1: The step definition is fixed within an experiment and logged with the run.

S2: Different step definitions are different trajectory representations and
    are analyzed separately, like different encoders (Spec 4.1 rules).

S3: Headline Atlas claims (motif existence, regime-outcome association) must
    replicate under at least two step definitions, or be explicitly reported
    as step-definition-specific.

S4: Features that mix scales (e.g., recurrence over long windows on token
    steps) must state the window in steps and in tokens, so results are
    comparable across definitions.

5. Fundamental Objects

5.1 State Space

$S \subseteq \mathbb{R}^d$

The operational latent state space produced by the selected encoder,
normalization (B3), and step definition (Section 4).

5.2 Trajectory

$\Gamma_i = (s_0, s_1, \ldots, s_T)$

A complete reasoning episode in latent state space.

5.3 Local Transition

$\delta_t = s_{t+1} - s_t$

A local displacement in the normalized latent space.

5.4 Trajectory Segment

$\Gamma_{t-w:t} = (s_{t-w}, \ldots, s_t)$

A local temporal window of length w used to characterize context-dependent
transitions. (v0.1 used k for the window; renamed to w to avoid collision

with the mode index k.)

5.5 Trajectory Features
---

r_t = r(Γ_(t-w:t))

Features and their discrete-space definitions (all on normalized states):

- transition magnitude: m_t = ||δ_t||
- velocity (mean speed):   (1/w) Σ ||δ_i|| over the window
- acceleration: second difference, ||δ_t - δ_(t-1)||,
                             averaged over the window
- directional consistency: mean pairwise cos(u_i, u_j) over the window
- directional diversity: V = 1 - ||(1/w) Σ u_i||
- curvature (discrete):   angle between successive unit transitions,
                             arccos(cos(u_t, u_(t-1))), averaged
- recurrence: max cosine of s_t to states before the window
- convergence rate: slope of ||δ_i|| over the window
- local dimensionality: participation ratio of the window's PCA spectrum
- spectral entropy: entropy of the normalized singular-value spectrum
                             of the windowed state matrix
- perturbation sensitivity: divergence of resampled continuations from the
                             same state (requires multiple seeds per state)
- coherence with task: cos(s_t, e_x) per Spec 8.2

Feature multicollinearity note: several features are strongly correlated by
construction. Analyses using r_t must report the feature correlation matrix
and use methods robust to collinearity; claims about individual features
require the others as covariates.

5.6 Functional Effect Profile

For a transition or trajectory segment, define:

```
e_i = (
    ΔBreadth,
    ΔCoherence,
    ΔUncertainty,
    ΔHypothesisQuality,
    ΔTestability,
    ΔOutcome,
    Persistence
)
```

Each component carries a metric-type label (B1):

- ΔBreadth, ΔCoherence, Persistence: Type G / Type E
- ΔHypothesisQuality, ΔTestability: Type J (judge separation B2 applies)
- ΔUncertainty: Type G (e.g., entropy of continuations)
                                     or Type J; the choice is fixed and logged
- ΔOutcome: Type O (hidden-solution tasks)

Functional effect profiles are required because geometric similarity alone
does not establish functional equivalence.

6. Operators as Equivalence Classes
===

The plan does not begin by assuming four operators.

Let:

D = {δ_1, δ_2, ..., δ_N}

be the observed transition set.

Define functional equivalence:

δ_i ~ δ_j

when, under comparable trajectory contexts, the two transitions:

- produce similar changes in trajectory geometry
- produce similar functional effect profiles
- retain similarity across prompts and domains
- have comparable downstream consequences

A discovered operator candidate is an equivalence class:

O_k = [δ]_k

or, more generally, a class of context-dependent local transformations:

O_k = {
    (Γ_context, δ)
    with recurring functional effect
}

Human labels such as EXPAND, CHALLENGE, CONNECT, and CONSOLIDATE may later be assigned for interpretability.

They are not axioms of the mature model.

7. Seed Intervention Families

The four existing operator families are retained only as data-generation instruments:

- EXPAND
- CHALLENGE
- CONNECT
- CONSOLIDATE

Additional trajectory sources must include:

- direct reasoning
- chain-of-thought
- self-reflection
- random operator schedules
- adversarial prompts
- analogy prompts
- contradiction prompts
- consolidation prompts
- unsuccessful reasoning
- successful reasoning
- random latent perturbations where technically safe

Purpose:

Generate broad trajectory diversity without forcing the final latent taxonomy to match the seed labels.

Coverage requirement (new in v0.2): at least 30% of Atlas episodes must come from non-operator sources (direct reasoning, plain chain-of-thought, random schedules), so that mode discovery has substantial data in which the seed taxonomy was never expressed. This supports the independence test of Section 9.5.

8. Phase A — Trajectory Atlas

8.1 Objective

Construct a large observational map of latent reasoning trajectories without assuming a final operator alphabet.

8.2 Dataset and Sizing

Use multiple domains, including:

- causal puzzles
- abductive reasoning
- scientific explanation
- hidden-rule discovery
- contradiction resolution
- design problems
- synthetic hypothesis reconstruction

Required variation:

- successful and unsuccessful episodes
- multiple seeds
- multiple prompt families

- multiple reasoning schedules
- matched compute budgets
- multiple problem difficulty levels

Sizing targets (new in v0.2; pilot tier):

- ≥ 4 domains, ≥ 50 problems per domain
- ≥ 5 seeds per problem-condition pair
- ≥ 10 trajectory sources per Section 7, with the 30% non-operator floor
- target: ≥ 10,000 episodes, ≥ 10^5 transitions at the reasoning-move step
  (correspondingly more at finer steps)

Rationale: cluster-stability analysis (Section 9.6) requires resampling; rule
of thumb is ≥ 100 segments per candidate mode per domain for cross-domain
recurrence tests. A pilot-based power analysis fixes confirmatory sizes,
per B4.

8.3 Recorded Data

For every episode, record:

- problem, domain, and difficulty level
- model and checkpoint
- state encoder
- selected layers
- normalization parameters (μ, W, calibration-set ID)
- step definition (Section 4) and window w
- full state sequence
- transition sequence
- prompt or intervention provenance (trajectory source, Section 7)
- operator seed label, if any
- decoded intermediate outputs
- final output
- objective outcome
- judge-based scores with judge identity (B2)
- realized compute, both ledgers (B4)
- episode length in steps and tokens

8.4 Analysis

The Trajectory Atlas should include:

- PCA and low-rank subspace analysis
- manifold visualization for exploration only
- clustering of transitions
- clustering of trajectory segments
- change-point detection
- recurrence analysis
- trajectory alignment
- dynamic time warping where appropriate
- spectral analysis
- local dimensionality estimation
- attractor-like region detection
- branching and convergence analysis
- outcome-conditioned trajectory comparison (with the controls of 8.6)

Visualization is exploratory.

All confirmatory claims must rely on held-out quantitative evaluation.

8.5 Primary Question

Do recurring trajectory motifs exist that:

- replicate across seeds
- survive cross-paraphrase evaluation
- survive cross-domain evaluation
- survive at least two step definitions (S3)
- predict functional effects
- predict downstream outcomes?

8.6 Outcome-Conditioning Confound Controls

(New in v0.2.)

Successful and unsuccessful episodes differ in trivial ways — length, difficulty, early stopping — and any of these can masquerade as a "motif of success". Required controls:

C1: Length matching. Outcome-conditioned comparisons are performed within strata of episode length (in steps and in tokens), or with length as an explicit covariate; a motif claimed to predict success must retain predictive value at fixed length.

C2: Difficulty stratification. Problems carry difficulty levels (8.2); outcome comparisons are made within difficulty strata. A motif that only separates easy from hard problems is a difficulty detector.

C3: Stopping-artifact check. Features computed near episode end (convergence, recurrence) are additionally evaluated on end-truncated trajectories to ensure they are not artifacts of the stopping rule.

C4: Domain balance. Outcome-conditioned motif claims must hold within at least two domains individually, not only in the pooled data.

8.7 Parallel Low-Cost Track: Seed-Operator Separability

(New in v0.2.)

The Atlas records seed labels (8.3), so the Spec's Experiment A — separability of EXPAND / CHALLENGE / CONNECT / CONSOLIDATE transitions, with cross-paraphrase and cross-domain splits — runs on Atlas data at near-zero marginal cost.

This track is run in parallel and reported regardless of Atlas outcomes:

- If separability holds, the discovery program proceeds with evidence that the state representation carries functional signal at all.
- If separability fails, Gate A failure semantics (Section 15) apply early, before major discovery investment.

This mitigates the main schedule risk of the trajectory-first ordering: remaining in exploratory analysis indefinitely.

9. Phase B — Functional Mode Discovery

9.1 Objective

Identify latent modes based on both trajectory geometry and functional consequence.

9.2 Functional Distance

Define:

```
d_functional(i, j)
    =
    α d_latent(i, j)
    +
    β d_effect(e_i, e_j)
    +
    γ d_context(c_i, c_j)
```

where:

d_latent:
distance between transition or trajectory representations

d_effect:
distance between functional effect profiles (component types per 5.6; Type J components obey B2)

d_context:
penalty for comparing non-equivalent initial states, operationalized as:

```
d_context(c_i, c_j)
    = 1 - cos(s_before_i, s_before_j)
      combined with a domain mismatch indicator
```

(other operationalizations are permitted but must be fixed and logged per experiment)

9.3 Weight Sensitivity Requirement

(New in v0.2.)

Discovered modes are a function of (α, β, γ). Therefore:

W1: Mode discovery is repeated over a pre-declared grid of (α, β, γ) (minimum: geometry-dominant, effect-dominant, and balanced settings).

W2: A mode is reportable only if it recurs — same membership up to a matching tolerance — across a contiguous region of the weight grid. Modes that exist at a single weight setting are artifacts of that setting.

W3: The weight grid and matching tolerance are fixed before confirmatory analysis (B4).

9.4 Discovery Methods

Candidate methods include:

- Gaussian mixture models
- spectral clustering
- hidden Markov models
- switching linear dynamical systems
- state-space models
- contrastive representation learning
- nonlinear mode discovery
- mixture-of-experts identification

No single clustering method is treated as ground truth.

Stable modes must appear under multiple reasonable analysis choices.

9.5 Seed-Label Independence

(New in v0.2.)

Because most Atlas trajectories are generated by prompted seed interventions, mode discovery risks merely recovering the seed taxonomy — a self-fulfilling result. Required tests:

I1: Mutual information between discovered mode assignments and seed labels is reported, with a permutation null. Near-perfect correspondence means the discovery added nothing beyond the seeds.

I2: A discovered mode is independently interesting if it (a) subdivides a seed family into functionally distinct sub-modes, (b) merges transitions across seed families with a shared effect profile, or (c) appears in non-operator-source trajectories (the ≥ 30% floor of Section 7).

I3: Confirmatory mode claims require recurrence in non-operator-source trajectories. Modes found only under seed prompting are classified as prompt-induced until shown otherwise.

9.6 Stability Criteria

A functional mode is considered provisionally stable when it shows:

- cluster stability under resampling
- cross-seed recurrence
- cross-paraphrase recurrence
- cross-domain recurrence
- recurrence across the weight grid (W2)
- presence in non-operator sources (I3)
- consistent effect profile
- predictive value for future trajectory evolution

If clusters track only prompt wording, domain, output length, or difficulty, they are artifacts (see 8.6).

10. Phase C — Causal Operator Construction

10.1 Objective

Convert stable functional modes into reusable causal transformations.

10.2 Candidate Operator Forms

Fixed vector:

$$T_k(s) = s + v_k$$

Linear operator:

$$T_k(s) = A_k s + b_k$$

State-dependent vector field:

$$T_k(s) = s + v_k(s)$$

Context-dependent operator:

$$T_k(s, \Gamma_{t-w:t})$$

Nonlinear operator:

$$T_k(s) = f_k(s)$$

10.3 Intervention Test

Given an unseen state s_t, apply via the Spec injection mechanism (B6):

$$s'_{t+1} = T_k(s_t, \Gamma_{t-w:t})$$

The operator succeeds only if the resulting trajectory produces the expected effect profile:

$$e(s'_{t+1}) \approx \text{mean\_effect}(O_k)$$

without an explicit textual instruction naming the function.

10.4 Required Controls

Compare discovered operators against:

- random directions
- norm-matched random directions
- PCA directions
- standard activation steering vectors
- prompt-only interventions
- shuffled mode labels
- no-intervention controls

10.5 Operator Validation

A candidate becomes a validated operator only if it demonstrates:

1. repeatability
2. cross-context transfer
3. causal behavioral effect
4. downstream utility
5. robustness to injection configuration
6. effect beyond prompt artifacts

11. Phase D — Trajectory Regime Discovery

11.1 Objective

Determine whether productive reasoning can be characterized by a low-dimensional trajectory regime.

11.2 Feature Vector

r(Γ) uses the features and definitions of Section 5.5.

The initial CLCD Ω metric is treated as one candidate summary, not as a fixed law.

11.3 Outcome Model

Learn:

$$P(Y_{good} \mid r(\Gamma))$$

where Y_good is primarily based on objective hidden-solution outcomes (Type O).

Secondary outcomes may include:

- hypothesis quality
- novelty
- testability
- explanatory power

Multiplicity control (new in v0.2): with ≥ 11 correlated features and flexible models, apparent predictive structure is easy to find. The outcome model is fit on a development split and evaluated once on a held-out confirmatory split; permutation-based feature importance with family-wise correction (B4); the confound controls of 8.6 (length, difficulty) enter as covariates.

11.4 Scalar or Vector Regime

If a stable low-dimensional scalar summary exists:

$$\Omega = f(r(\Gamma))$$

it may be retained as the trajectory regime metric.

If no stable scalar exists, use a multidimensional regime vector:

```
Ω_vec = (
    exploration,
    stability,
    coherence,
    plasticity,
    convergence
)
```

Status note (new in v0.2): the Ω_vec component names are placeholders. Each must be defined as an explicit function of r(Γ) before use; undefined named dimensions may not appear in any analysis.

The theory does not require a single scalar if the data do not support one.

11.5 Productive Zone Test

Test whether:

$$P(Y_{good} \mid \Omega \text{ in productive region}) > P(Y_{good} \mid \Omega \text{ outside productive region})$$

The productive region must be selected on development data and evaluated once on held-out data, consistent with the Spec's pre-registration rules (Ω1-Ω3).

12. Phase E — Closed-Loop Control

12.1 Objective

Use validated operators to control trajectory regimes dynamically.

12.2 Controller State

```
z_t = (
    current trajectory features,
    recent mode history,
    operator history,
    task progress,
    compute consumed,
    current regime estimate
)
```

12.3 Regime Target Definition and Data Splits

(New in v0.2. v0.1 used a desired regime r* without defining its origin.)

The desired regime r_t* is derived from the Phase D productive region: it is the point (or set) in regime space associated with highest P(Y_good) on the Phase D development split.

Leakage rule L1: the data used to (a) discover the productive region, (b) train the controller, and (c) evaluate the controller must be three disjoint splits. If the regime target and the controller policy are derived from the same episodes that later score the controller, the evaluation is circular.

Leakage rule L2: judges used in controller training obey B2 with respect to outcome evaluation.

12.4 Control Objective

The controller does not select human labels as its final abstraction.

It selects the transformation expected to move the future trajectory toward the productive regime:

```
T_t*
    =
argmin over T_k
E[
    d(
        r(Γ_(t+1)),
        r_t*
    )
]
```

12.5 Trajectory Functional

Define a trajectory-level objective:

```
J[Γ]
    =
    Q_outcome(Γ)
    - λ_1 Compute(Γ)
    - λ_2 Drift(Γ)
    - λ_3 Repetition(Γ)
    + λ_4 ProductiveReorganization(Γ)
```

The controller seeks:

```
max over operator sequence
E[J[Γ]]
```

12.6 Controller Comparisons

Compare:

- no controller
- fixed operator cycle
- random operator policy
- rule-based CLCD controller (from the Spec)
- learned mode controller
- oracle controller for upper-bound estimation

All comparisons require equal realized compute and equal tuning budgets (B4, B5).

13. Additional Related Work

(New in v0.2. Supplements Spec Section 3; same citation-verification rule.)

13.1 Options and skill discovery in hierarchical RL

Phases B-C-E are structurally an options / skill-discovery program: discover reusable temporally-extended transformations from experience, then learn a policy over them. Directly relevant lines:

- options framework (Sutton, Precup, Singh 1999)
- unsupervised skill discovery via mutual-information objectives (e.g., DIAYN — Eysenbach et al. 2019)
- successor features and transferable behaviors

What CLCD adds: the "environment" is the model's own latent reasoning space; skills act on activations rather than an external world; and skill validity is defined by functional effect profiles on reasoning outcomes. The RL literature supplies discovery objectives (mutual information between skill and visited states) that can be reused in Phase B.

13.2 Latent dynamics methods from computational neuroscience

The Phase B toolset (switching linear dynamical systems, state-space models, HMM segmentation of latent trajectories) is standard in neural population dynamics analysis (e.g., Linderman et al. on switching LDS; low-dimensional dynamics in motor cortex). This literature also documents the field's main pitfall — descriptive dynamical structure without causal validation — which is exactly the Gate B / Gate C distinction of this plan.

[TODO: full citation pass before external circulation, per Spec 3.5 rule.]

14. Experimental Sequence

Experiment 0:
Trajectory Atlas

Question:
Does recurring structure exist?

Experiment 0b (parallel, low cost — Section 8.7):
Seed-Operator Separability on Atlas data

Question:
Does the state representation carry functional signal at all?

Experiment 1:
Functional Mode Discovery

Question:
Do recurring structures correspond to stable functional effects?

Experiment 2:
Causal Mode Intervention

Question:
Can stable modes be converted into causal operators?

Experiment 3:
Regime Discovery

Question:
Can trajectory geometry predict reasoning outcomes?

Experiment 4:
Closed-Loop Control

Question:
Can a controller use discovered operators to improve reasoning under matched compute?

15. Decision Gates

Gate A:
No reproducible trajectory structure.

Failure semantics (extended in v0.2): a negative Atlas result cannot by
itself distinguish (i) absence of structure from (ii) an inadequate encoder,
normalization, or step definition. Gate A is only conclusive after the
robustness sweep: ≥ 2 encoders, ≥ 2 normalizations (B3), ≥ 2 step
definitions (S3). If the sweep was not completed, the consequence is
"representation revision", not "hypothesis unsupported".

Consequence (after a completed sweep):
The operator hypothesis is not supported under the tested state
representations.

Gate B:
Geometric clusters exist but lack stable functional effects.

Consequence:
The clusters are descriptive geometry, not functional operators.

Gate C:
Functional modes exist but cannot be causally applied.

Consequence:
The modes are correlational markers, not controllable operators.
(Spec 6.4 applies: failed injection-validity checks indict the encoder /
injection pair, not the modes.)

Gate D:
Operators work but no stable productive regime exists.

Consequence:
Use direct outcome-based control; remove Ω from the core architecture.

Gate E:
Regimes exist but controller does not improve outcomes.

Consequence:
Reject or revise closed-loop operator selection.

Gate F:
Controller gains disappear under matched compute.

Consequence:
Report orchestration overhead rather than reasoning improvement.

16. Falsification Conditions

The trajectory-first CLCD program is weakened if:

1. latent trajectories are not reproducible under matched conditions

2. discovered modes fail cross-domain and cross-paraphrase tests

3. functional effects do not align with geometric modes

4. mode identity depends primarily on prompt wording (I1-I3)

5. causal interventions fail despite valid injection

6. discovered transformations do not transfer

7. trajectory features do not predict objective outcomes after length and
   difficulty controls (8.6)

8. controller policies do not outperform random or fixed schedules

9. all gains are explained by extra compute

10. simpler established steering or search methods fully explain the results

17. Relationship to the Spec (CLCD Architecture Specification v0.2)

The Spec remains valid as an implementation framework.

The trajectory-first plan changes the epistemic status of several components.

Previous status:

EXPAND, CHALLENGE, CONNECT, CONSOLIDATE
    =
candidate operator alphabet

Revised status:

EXPAND, CHALLENGE, CONNECT, CONSOLIDATE
    =
seed intervention families

Previous status:

Ω
    =
defined trajectory regime metric

Revised status:

Ω
    =
candidate summary to be compared against learned regime representations

Previous status:

Operator Discovery Experiment
    =
first experiment

Revised status:

Trajectory Atlas (with parallel Experiment 0b)
    =
first experiment

All Spec protocols listed in Section 3 (B1-B6) remain binding.

18. Minimum Trajectory Atlas Prototype
=====================================

The minimum prototype requires:

1. one open-weight LLM with hidden-state access

2. one fixed state encoder and two normalization variants (B3)

3. two step definitions (S3)

4. at least three reasoning domains

5. successful and unsuccessful reasoning episodes with difficulty labels

6. multiple prompt families and seeds; ≥ 30% non-operator sources
   (Section 7)

7. full trajectory logging per 8.3

8. objective hidden-solution outcomes

9. exploratory trajectory analysis

10. held-out tests for recurring motifs

11. Experiment 0b (seed-operator separability) on the same data

12. no claim of causal operators at this stage

19. Immediate Actions
=====================

1. Freeze the Spec as the current architecture reference; adopt B1-B6 as
   binding for this plan.

2. Treat the four named operators as seed interventions only.

3. Select the first open-weight model.

4. Fix two step definitions per Section 4 (recommended: ST-2 and ST-3).

5. Define the state extraction layer and pooling method; freeze
   normalization on a calibration split.

6. Create the first hidden-solution task set with difficulty labels.

7. Generate matched successful and unsuccessful trajectories with length
   and difficulty stratification (8.6).

8. Build the Trajectory Atlas logging schema (8.3), including the ≥ 30%
   non-operator source floor.

9. Run Experiment 0b (separability) as soon as Atlas data exists.

10. Run exploratory low-rank, recurrence, and change-point analyses.

11. Test motif stability across seeds, prompts, domains, step definitions,
    and the (α, β, γ) weight grid.

12. Run the seed-label independence tests (9.5) on any candidate modes.

13. Do not train a controller before functional modes are validated.

14. Do not claim operators before causal intervention succeeds.

20. Current Central Claim
==========

The strongest defensible CLCD claim at this stage is:

Machine reasoning may generate structured latent trajectories containing
recurring functional modes.

If those modes can be discovered, causally reproduced, and selected through
closed-loop control, they may form the basis of a new latent-state
architecture for machine reasoning and hypothesis generation.

End of CLCD Trajectory-First Research Plan v0.2
==========